Test Plan

Swag Labs (SauceDemo) - Manual Functional Testing

Item	Details
Product	Swag Labs sample e-commerce application (https://www.saucedemo.com)
Document	Test Plan
Version / Status	1.0 / Draft
Author	Hilary (QA Engineer)
Date	27 October 2026

1. Objective

This plan defines the scope, approach, schedule and deliverables for manual functional testing of the Swag Labs web application. It is the basis for the test cases, defect reports and test summary produced in this project, and it demonstrates a structured QA process for portfolio purposes.

2. Reference Documents

Document	Location / Identifier
Functional Requirements Specification (mini FRS)	Requirements FR-1.1 to FR-7.2, derived from observing the application (no official specification exists)
Test cases and execution results	test-cases/ folder in the project repository (TC_SwagLabs_001 onward)
Defect tracker	GitHub Issues in the saucedemo-qa-portfolio repository
Application under test	https://www.saucedemo.com

3. Scope

3.1 Areas to be tested

Area	Requirements	What is covered
Login	FR-1.1 to FR-1.5	Valid and invalid credentials, empty fields, locked-out user, access to pages without logging in
Products page	FR-2.1	Product list and the details shown on each product
Product details	FR-2.2, FR-2.3	Opening an item from its image or title, returning to the products page
Sort dropdown	FR-6.1	Name (A to Z, Z to A) and price (low to high, high to low)
Cart	FR-3.1 to FR-3.5	Adding and removing items from the products and details pages, cart persistence on refresh

Area	Requirements	What is covered
Cart page	FR-4.1 to FR-4.6	Cart badge count, cart contents, continue shopping, removing items, starting checkout, empty cart
Checkout	FR-5.1 to FR-5.5	Customer information validation, order totals and tax, completing the order, canceling, returning home
Logout	FR-7.1	Logging out and session end
Reset App State	FR-7.2	Clearing the cart from the menu

3.2 Areas not in scope

- Performance and load testing
- Security and penetration testing (basic access-control checks, such as opening a page without logging in, are in scope)
- Accessibility testing
- Mobile, tablet and responsive layout testing
- Internationalisation and localisation
- Visual or pixel-level design checks
- Footer social media links, the About page and any external sites
- Dynamic Catalog
- Special demo users other than standard_user and locked_out_user (for example problem_user and performance_glitch_user)

4. Test Approach

Test type	In scope	How it is applied
Functional (positive)	Yes	Each requirement is verified with valid input and expected user actions
Negative	Yes	Invalid, empty and unauthorised actions are tried for every feature that accepts input
Boundary	Yes	Smallest and largest valid input for text fields, using equivalence partitioning and boundary value analysis
Exploratory	Yes	Short, time-boxed sessions for behaviour the requirements do not describe, such as session handling
Regression	Yes	High-priority cases are re-run after any change to the application or any defect reported as fixed
Cross-browser	Yes	Smoke check of High-priority cases on additional browsers (see section 6)
Integration, acceptance, performance, localisation	No	Not applicable to this single-application, solo project (see 3.2)

5. Test Strategy

- **Requirement-based:** every test case traces to a requirement ID in the mini FRS, and every requirement must have at least one test case.
- **Priority-based execution:** High-priority cases run first, then Medium, then Low. If time runs short, Low-priority cases are the ones deferred.
- **Results recorded for every case:** Actual Result and Status (Passed, Failed, Blocked or Not Run) are filled in during execution. Expected results are never edited to match actual results.
- **Defects:** each failed case is reported on GitHub Issues with the test case ID, or recorded as a known issue with a reason (see section 9). A failure is reproduced at least twice before it is reported.
- **Unclear requirements:** when the requirements are silent, the observed behaviour is recorded and raised as a question rather than reported as a defect.
- **Retest:** a defect reported as fixed is retested, followed by a regression run of High-priority cases.

6. Test Environment and Test Data

Item	Details
Application URL	https://www.saucedemo.com
Operating system	Ubuntu 24 LTS
Test accounts	standard_user (normal user) locked_out_user (locked-out behaviour) Password for both: secret_sauce (published on the login page)
Test data	The six listed products with their prices; sample customer details (name and postal code) for checkout

Browser	Version	Coverage
Chrome	Recorded at execution	Full test suite
Edge	Recorded at execution	High-priority cases
Firefox	Recorded at execution	High-priority cases

7. Automation Strategy

Automation starts after the first manual execution cycle is complete.

- **Tool:** Playwright with TypeScript/JavaScript.
- **Scope:** stable, High-priority flows: login (valid and invalid), add to cart, cart badge, checkout validation and the end-to-end purchase.
- **Design:** Page Object Model, with test data kept separate from test logic.
- **Execution:** tests run on GitHub Actions for every push and pull request, with the HTML report saved as an artifact.
- **Stays manual:** exploratory sessions, session-handling observations and any case that depends on judgement.

8. Schedule

Phase	Activity	Planned period
1	Requirements analysis and mini FRS	Month 1, Weeks 2-3
2	Test case design and review	Month 1, Week 3
3	Test execution (cycle 1)	Month 1, Week 3
4	Defect reporting on GitHub Issues	Month 1, Week 4
5	Test plan finalised and test summary written	Month 1, Week 4
6	Automation of High-priority cases	Month 2 onward

9. Defect Management

Defects are logged as GitHub Issues in the project repository, using the labels bug and one of severity: high, severity: medium or severity: low. Each report contains the environment, numbered steps to reproduce, expected result, actual result, evidence (screenshot or recording) and the related test case ID.

Severity	Definition	Example
High	A core flow is blocked, or money or security is affected	Order cannot be completed; a protected page opens without login
Medium	A feature behaves incorrectly but the user can still complete the main task	Checkout is allowed with an empty cart
Low	Cosmetic or minor inconsistency	A button label stays stale until the page is refreshed

Defect lifecycle: New, Open, Fixed, Retest, then Closed (or Reopened if the fix fails).

10. Test Deliverables

- Test Plan (this document)
- Mini FRS (requirements)
- Test cases with execution results
- Defect reports on GitHub Issues
- Test summary report: cases executed, pass rate, defects by severity, and any open or known issues

11. Resources

Role / Resource	Details
QA Engineer	Hilary: test design, execution, defect reporting and automation
Hardware and OS	Ubuntu 24 LTS Machine

Role / Resource	Details
Software and tools	Chrome, Edge, Firefox; spreadsheet for test cases; GitHub (repository and Issues); Node.js and Playwright (automation phase)

12. Risks and Mitigation

No.	Risk	Mitigation
1	No official requirements exist for the application	Requirements are inferred from observed behaviour and documented in the mini FRS; ambiguities are raised as questions
2	The third-party demo site may change, go down or reset without notice	Record the date and browser for every execution run and keep screenshot evidence; re-run affected cases if the site changes
3	Unexpected logouts during execution	Restart the affected test from its preconditions; note the idle time; confirm with a clean browser profile before reporting
4	A single tester may miss cases or share the author's blind spots	Check coverage against the FRS (Linked Cases count); run time-boxed exploratory sessions; ask for a peer review
5	Expected results are based on assumed behaviour and may be wrong	Review every mismatch as either a defect or a requirement correction; never edit Expected to match Actual
6	Loss of work or files	All documents and test cases are kept in the GitHub repository

13. Entry Criteria

- Mini FRS completed and reviewed
- Test plan reviewed
- Test cases written, prioritised and linked to requirement IDs
- Application reachable and test accounts able to log in
- Browsers installed and GitHub repository set up with labels

14. Exit Criteria

- All High-priority cases executed; all Medium and Low cases executed or deferred with a recorded reason
- Every failed case has a defect report or is documented as a known issue
- No cases left in Blocked status without an explanation
- Test summary report completed
- All deliverables committed to the repository